

Manipulação e Organização de Dados

Da base bruta à base analítica com Pandas

Aléssio Tony Cavalcanti de Almeida

Centro de Ciências Sociais Aplicadas
Universidade Federal da Paraíba
alessio@lema.ufpb.br



CIÊNCIA DE
DADOS PARA
NEGÓCIOS

João Pessoa, 2026

Plano de aula

Objetivo: desenvolver a capacidade de transformar dados brutos, dispersos e inconsistentes em uma **base analítica organizada**, adequada para descrição, visualização, modelagem e tomada de decisão.

Competências técnicas

- Identificar unidade de observação e chave.
- Padronizar nomes, tipos e formatos.
- Concatenar bases equivalentes.
- Integrar fontes com merge.

Competências analíticas

- Auditar perdas, duplicidades e ausências.
- Agregar dados em indicadores.
- Reorganizar dados em formato longo e largo.
- Entregar evidências em tabela final.

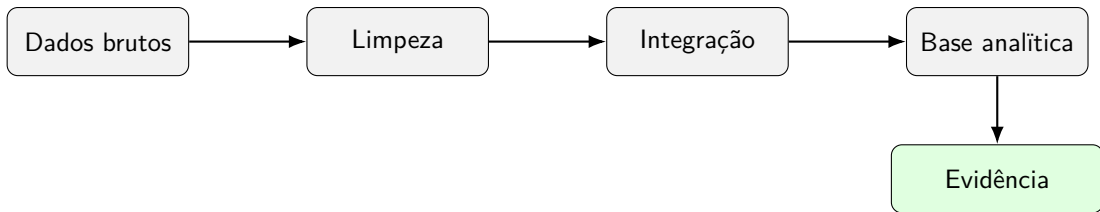
Estrutura da aula

Onde estamos na aula?

Por que manipular e organizar dados?

Pergunta orientadora

Como sair de planilhas, arquivos e tabelas dispersas para uma base pronta para responder uma pergunta de negócio ou de política pública?



Mensagem central

Manipulação de dados não é apenas aplicar comandos. **É desenhar uma representação correta do problema analítico.**

Caso guia da aula

Pergunta orientadora

Quais municípios combinam **alta taxa de abandono no ensino médio público** e **baixa adequação da formação docente**?

Bases usadas no exemplo didático

- **Rendimento escolar:** ano, município, matrículas e taxa de abandono.
- **Adequação docente:** ano, município e percentual de docentes com formação adequada.
- **População:** município e população de referência.

Produto final esperado

Uma tabela de evidências com municípios prioritários, indicadores calculados e diagnóstico de qualidade da integração.

Antes do Pandas: cinco decisões analíticas

- ❶ **Pergunta:** qual problema será respondido?
- ❷ **Unidade de observação:** o que representa uma linha?
- ❸ **Chave:** qual combinação identifica uma linha de forma única?
- ❹ **Granularidade:** escola, município, estado, ano, turma, aluno?
- ❺ **Produto:** tabela, painel, modelo, mapa, relatório ou indicador?

Regra prática

Se a chave está errada, o merge, o groupby e o modelo também estarão.

Onde estamos na aula?

Aquecimento diagnóstico: o que acontece no join?

Tabela A: municípios

id	município	uf
1	São Paulo	SP
2	Rio de Janeiro	RJ
3	Belo Horizonte	MG
4	João Pessoa	PB

Tabela B: população

id	população
1	12.300.000
2	6.748.000
4	817.511

Antes de executar no notebook

Preveja quantas linhas existirão em cada resultado: inner, left, right e outer. Justifique em uma frase.

PRIMM 1: prever e executar

```
municipio = pd.DataFrame({
    "id": [1, 2, 3, 4],
    "nome": ["Sao Paulo", "Rio de Janeiro", "Belo Horizonte", "Joao
                                                    Pessoa"],
    "uf": ["SP", "RJ", "MG", "PB"]})

populacao = pd.DataFrame({
    "id": [1, 2, 4],
    "populacao": [12300000, 6748000, 817511]})

pd.merge(municipio, populacao, on="id", how="left")
```

Roteiro PRIMM

Predict: quantas linhas? **Run:** execute. **Investigate:** onde apareceu NaN? **Modify:** troque para inner. **Make:** crie um exemplo com um município extra na população.

Onde estamos na aula?

O que representa uma linha?

Base	Unidade de observação	Chave esperada
Microdados do ENEM	estudante-inscrição	ano + nu_inscricao
Censo Escolar	escola-ano	ano + id_escola
Indicador municipal	município-ano-indicador	ano + id_mun + indicador
Compras públicas	item-licitação	id_licitacao + id_item
Série financeira	ativo-data	ticker + data

Atividade em sala

Para cada exemplo, diga se seria correto fazer um merge usando apenas o código territorial.

Criando as bases do caso guia

```
import pandas as pd
import numpy as np

rendimento_2022 = pd.DataFrame({
    "ano": [2022, 2022, 2022, 2022],
    "id_mun": [2507507, 2510808, 2504009, 2503209],
    "municipio": ["Joao Pessoa", "Patos", "Campina", "Cabedelo"],
    "uf": ["PB", "PB", "PB", "PB"],
    "matriculas_em": [18500, 4200, 13200, 3100],
    "tx_abandono_em": [5.8, 8.2, 6.1, 4.9]})

rendimento_2023 = pd.DataFrame({
    "ano": [2023, 2023, 2023, 2023],
    "id_mun": [2507507, 2510808, 2504009, 2503209],
    "municipio": ["Joao Pessoa", "Patos", "Campina", "Cabedelo"],
    "uf": ["PB", "PB", "PB", "PB"],
    "matriculas_em": [18700, 4100, 13500, 3300],
    "tx_abandono_em": [5.2, 9.0, 5.7, 4.5]})
```

Criando bases complementares

```
docentes = pd.DataFrame({
    "ano": [2022, 2022, 2022, 2022, 2023, 2023, 2023],
    "id_mun": [2507507, 2510808, 2504009, 2503209,
               2507507, 2510808, 2504009],
    "perc_docentes_adequados": [72.4, 61.2, 75.8, 70.1,
                                 74.0, 58.5, 77.2]
})

populacao = pd.DataFrame({
    "id_mun": [2507507, 2510808, 2504009, 2503209],
    "pop_2022": [833932, 108192, 419379, 69673]
})
```

Alerta

A base de docentes não possui Cabedelo em 2023. Isso permite discutir cobertura, ausência e auditoria de merge.

Inspeção inicial: tamanho, tipos e amostra

```
rendimento_2022.shape  
rendimento_2022.head()  
rendimento_2022.info()  
rendimento_2022.describe(include="all")
```

O que verificar antes de qualquer transformação?

- Existem colunas com tipo errado?
- A chave candidata possui duplicidades?
- Há valores ausentes em campos essenciais?
- Os nomes das colunas são legíveis e consistentes?

Validação da chave

```
chave = ["ano", "id_mun"]

rendimento_2022.duplicated(chave).sum()
docentes.duplicated(chave).sum()
```

Interpretação

- `duplicated(chave).sum() == 0`: a chave identifica unicamente as linhas.
- `duplicated(chave).sum() > 0`: há risco de multiplicação artificial no merge.

Atividade em sala

O que aconteceria se a chave fosse apenas `id_mun`?

Dados organizados: formato analítico

Estrutura recomendada

- 1 Cada variável forma uma coluna.
- 2 Cada observação forma uma linha.
- 3 Cada tipo de unidade observacional forma uma tabela.

Formato longo

Melhor para gráficos, modelos, painéis e indicadores repetidos no tempo.

Formato largo

Melhor para comparação direta, planilhas finais e algumas matrizes de modelagem.

Onde estamos na aula?

Padronização de nomes de colunas

```
# Exemplo generico para bases reais com nomes problematicos
def limpar_colunas(df):
    return (
        df.rename(columns=lambda x: x.strip()
                    .lower()
                    .replace(" ", "_")
                    .replace("-", "_")
                    .replace("/", "_")))

rendimento_2022 = limpar_colunas(rendimento_2022)
rendimento_2023 = limpar_colunas(rendimento_2023)
```

Por que padronizar?

Evita erros silenciosos por acentos, espaços, diferenças de capitalização e nomes incompatíveis entre arquivos.

Conversão de tipos: strings, números e datas

```
dados = pd.DataFrame({
    "nome": ["Alice", "Bruno", "Carla", "David"],
    "idade": ["25", "32", "40", "22"],
    "salario": ["4.500,50", "5.200,75", "6.300,00", "3.800,00"],
    "data_contratacao": ["12/05/2022", "15/08/2021", "10/10/2020", "20
                        /11/2023"]
})

dados["idade"] = dados["idade"].astype(int)
dados["salario"] = (
    dados["salario"]
    .str.replace(".", "", regex=False)
    .str.replace(",", ".", regex=False)
    .astype(float)
)

dados["data_contratacao"] = pd.to_datetime(
    dados["data_contratacao"], format="%d/%m/%Y")
```

Criando variáveis com assign

```
dados = (
    dados
    .assign(
        ano_contratacao=lambda df: df["data_contratacao"].dt.year,
        salario_mil=lambda df: df["salario"] / 1000,
        faixa_idade=lambda df: pd.cut(
            df["idade"],
            bins=[0, 29, 39, 120],
            labels=["ate_29", "30_39", "40_mais"]
        )
    )
)
```

Boa prática

Prefira encadear transformações quando isso torna o fluxo mais legível e reproduzível.

Tratamento de ausentes: remover, imputar ou sinalizar?

```

painel_tmp = pd.DataFrame({
    "id_mun": [1, 2, 3, 4],
    "tx_abandono": [5.1, np.nan, 8.4, 4.0],
    "matriculas": [1000, 850, 1200, np.nan]})

painel_tmp.isna().sum()
# 1. Remover linhas com ausentes em variaveis essenciais
painel_sem_na = painel_tmp.dropna(subset=["tx_abandono"])
# 2. Imputar com mediana quando fizer sentido analitico
painel_tmp["matriculas"] = painel_tmp["matriculas"].fillna(
    painel_tmp["matriculas"].median())
# 3. Criar flag de ausencia
painel_tmp["flag_tx_abandono_ausente"] = painel_tmp["tx_abandono"].
    isna()
    
```

Onde estamos na aula?

Quando usar concat?

Ideia

Concatenar significa combinar objetos ao longo de um eixo. Em projetos de dados, o caso mais comum è **empilhar arquivos de anos, meses ou unidades com a mesma estrutura**.

Situação	Exemplo	Eixo
Novas linhas	RAIS 2021 + RAIS 2022 + RAIS 2023	axis=0
Novas colunas	indicadores calculados em tabelas alinhadas	axis=1
Painel temporal	rendimento escolar por ano	axis=0

Cuidado

Antes de empilhar, verifique se as colunas possuem o mesmo significado, tipo e escala.

Concatenação no caso guia

```
rendimento = pd.concat(  
    [rendimento_2022, rendimento_2023],  
    ignore_index=True  
)  
  
rendimento.sort_values(["id_mun", "ano"])
```

Resultado esperado

A base passa a ter uma estrutura de painel: cada linha corresponde a um **município-ano**.

Auditoria depois da concatenação

```
# Numero de linhas por ano
rendimento["ano"].value_counts().sort_index()

# Duplicidades na chave do painel
rendimento.duplicated(["ano", "id_mun"]).sum()

# Cobertura de municipios por ano
rendimento.groupby("ano")["id_mun"].nunique()
```

Atividade em sala

Se um ano tiver menos municípios do que os demais, isso é necessariamente um erro? Como diagnosticar?

Exercício rápido: empilhar bases

Atividade em sala

Crie uma base rendimento empilhando rendimento_2022 e rendimento_2023. Depois, responda:

- ① Quantas linhas existem por ano?
- ② A chave ano + id_mun é única?
- ③ Qual município apresentou maior taxa de abandono em cada ano?

```

rendimento = pd.concat([rendimento_2022, rendimento_2023],
                        ignore_index=True)

rendimento.loc[
    rendimento.groupby("ano")["tx_abandono_em"].idxmax(),
    ["ano", "municipio", "tx_abandono_em"]
]
```

Onde estamos na aula?

Quando usar merge?

Ideia

Junção combina tabelas a partir de uma ou mais chaves comuns. É a operação central para enriquecer uma base principal com informações complementares.

Tipo	Interpretação
left	Mantêm todas as linhas da base principal. Útil para enriquecer a base de análise.
inner	Mantêm apenas chaves presentes nas duas bases. Pode remover observações silenciosamente.
outer	Mantêm todas as chaves das duas bases. Útil para auditoria de cobertura.
right	Mantêm todas as linhas da base da direita. Menos usado em pipelines.

Junção simples com auditoria

```
painel = rendimento.merge(  
    docentes,  
    on=["ano", "id_mun"],  
    how="left",  
    validate="one_to_one",  
    indicator=True  
)  
  
painel["_merge"].value_counts()
```

Por que usar validate e indicator?

- validate="one_to_one": impede multiplicação inesperada de linhas.
- indicator=True: registra se cada linha veio de ambas as bases ou apenas de uma.

Identificando perdas ou ausências no merge

```
sem_docentes = (  
    painel  
    .query("_merge == \"left_only\"")  
    [["ano", "id_mun", "municipio", "tx_abandono_em"]]  
)  
  
sem_docentes
```

Interpretação

A ausência de informação complementar não deve ser escondida. Ela deve ser documentada e, quando relevante, transformada em indicador de qualidade da base.

Junção com chaves de nomes diferentes

```
empresas = pd.DataFrame({  
    "id_empresa": [1, 2, 3],  
    "empresa": ["ABC", "InovTec", "FoodTech"]  
})  
  
vendas = pd.DataFrame({  
    "codigo_empresa": [2, 3, 4],  
    "vendas": [90, 85, 75]  
})  
  
empresas.merge(  
    vendas,  
    left_on="id_empresa",  
    right_on="codigo_empresa",  
    how="left",  
    indicator=True  
)
```


Junção com chave composta

```
lojas = pd.DataFrame({
    "id_empresa": [1, 1, 2],
    "id_filial": [10, 11, 20],
    "empresa": ["ABC", "ABC", "FoodTech"]
})
```

```
vendas_filial = pd.DataFrame({
    "id_empresa": [1, 2, 4],
    "id_filial": [11, 20, 40],
    "vendas": [90, 85, 75]
})
```

```
lojas.merge(
    vendas_filial,
    on=["id_empresa", "id_filial"],
    how="left",
    validate="one_to_one",
    indicator=True
)
```

Erros comuns em junções

Erro	Consequência	Prevenção
Chave incompleta	Duplica linhas ou combina anos diferentes	Definir unidade de observação
Tipos incompatíveis	Chaves não casam	Converter tipos antes do merge
inner sem auditoria	Remove observações	Usar indicator=True
Muitos-para-muitos oculto	Infla totais e indicadores	Usar validate
Nomes ambíguos	Colunas _x e _y sem controle	Usar suffixes ou renomear

Exercício rápido: escolha do join

Atividade em sala

A base de rendimento tem 5.570 municípios, mas a base de adequação docente tem 5.430. Qual join usar?

- ① inner: bom para manter apenas municípios com dados completos, mas reduz cobertura.
- ② left: bom quando rendimento é a base principal.
- ③ outer: bom para auditoria geral de cobertura.

```

auditoria = rendimento.merge(
    docentes, on=["ano", "id_mun"],
    how="outer", indicator=True)

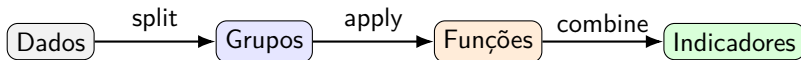
auditoria["_merge"].value_counts()
  
```

Onde estamos na aula?

O que groupby faz?

Lógica *split-apply-combine*

- 1 **Split:** divide os dados em grupos.
- 2 **Apply:** aplica uma função a cada grupo.
- 3 **Combine:** combina os resultados em uma tabela final.



Criando variáveis antes de agregar

```

painel = (
    painel
    .drop(columns="_merge")
    .assign(
        abandono_estimado=lambda df: df["matriculas_em"] * df["
                                     tx_abandono_em"] / 100,
        alto_abandono=lambda df: df["tx_abandono_em"] >= 7,
        baixa_adequacao_docente=lambda df: df["perc_docentes_adequados
                                             "] < 65
    )
)
    
```

Decisão substantiva

alto_abandono e baixa_adequacao_docente dependem de limiares. Esses limiares devem ser justificados pelo contexto ou por critérios estatísticos.

Agregação por ano

```
resumo_anual = (  
    painel  
    .groupby("ano", as_index=False)  
    .agg(  
        municipios=("id_mun", "nunique"),  
        matriculas_em=("matriculas_em", "sum"),  
        abandono_estimado=("abandono_estimado", "sum"),  
        media_tx_abandono=("tx_abandono_em", "mean"),  
        media_adequacao_docente=("perc_docentes_adequados", "mean")  
    )  
)  
  
resumo_anual
```

Média simples x média ponderada

```
media_simples = painel["tx_abandono_em"].mean()

media_ponderada = np.average(
    painel["tx_abandono_em"],
    weights=painel["matriculas_em"]
)

media_simples, media_ponderada
```

Interpretação

A média simples descreve o **município médio**. A média ponderada por matrículas aproxima melhor a experiência do **estudante médio**.

Função auxiliar para média ponderada

```
def media_ponderada(x, pesos):
    mascara = x.notna() & pesos.notna()
    return np.average(x[mascara], weights=pesos[mascara])

resumo_anual_pond = (
    painel
    .groupby("ano")
    .apply(lambda g: pd.Series({
        "tx_abandono_media_pond": media_ponderada(
            g["tx_abandono_em"], g["matriculas_em"]
        ),
        "matriculas_em": g["matriculas_em"].sum(),
        "abandono_estimado": g["abandono_estimado"].sum()
    }))
    .reset_index()
)
```

Tabela de evidências

```

evidencias = (
    painel
    .query("alto_abandono == True or baixa_adequacao_docente == True")
    .sort_values(["ano", "tx_abandono_em"], ascending=[True, False])
    [[
        "ano", "municipio", "matriculas_em", "tx_abandono_em",
        "perc_docentes_adequados", "abandono_estimado",
        "alto_abandono", "baixa_adequacao_docente"
    ]]
)

evidencias
  
```

Produto analítico

A tabela final deve permitir decisão: priorizar territórios, investigar causas ou desenhar intervenções.

Limitações comuns da agregação

- **Perda de heterogeneidade:** médias escondem desigualdades internas.
- **Peso populacional:** municípios pequenos e grandes podem receber o mesmo peso.
- **Mudança de granularidade:** conclusões municipais não valem automaticamente para escolas ou estudantes.
- **Viés de cobertura:** ausências podem afetar a comparação entre grupos.
- **Ecological fallacy:** relações agregadas podem não refletir relações individuais.

Onde estamos na aula?

Formato longo x formato largo

Formato longo

mun	ano	valor
A	2022	5.1
A	2023	4.8
B	2022	8.2
B	2023	9.0

Uso: gráficos, modelos, painéis, groupby.

Formato largo

mun	2022	2023
A	5.1	4.8
B	8.2	9.0

Uso: comparação direta, matriz, exportação.

De longo para largo: pivot_table

```
abandono_wide = painel.pivot_table(  
    index=["id_mun", "municipio"],  
    columns="ano",  
    values="tx_abandono_em"  
) .reset_index()  
  
abandono_wide
```

Quando usar?

Quando queremos comparar anos em colunas ou preparar uma matriz de variáveis.

De largo para longo: melt

```
abandono_long = abandono_wide.melt(  
    id_vars=["id_mun", "municipio"],  
    var_name="ano",  
    value_name="tx_abandono_em"  
)  
  
abandono_long
```

Quando usar?

Quando precisamos voltar para uma estrutura compatível com gráficos, painéis, filtros e modelos.

Exercício rápido: variação temporal

Atividade em sala

A partir da tabela larga, calcule a variação da taxa de abandono entre 2022 e 2023.

```
abandono_wide = abandono_wide.assign(
    variacao_2023_2022=lambda df: df[2023] - df[2022]
)

abandono_wide.sort_values("variacao_2023_2022", ascending=False)
```

Discussão

Uma variação positiva indica piora. Mas a interpretação depende de cobertura, escala, tamanho da rede e erro de mensuração.

Onde estamos na aula?

Anonimização também é organização de dados

Objetivo

Reduzir o risco de identificação de indivíduos, preservando a utilidade analítica da base.

Técnica	Exemplo
Remoção	Excluir CPF, nome, telefone e endereço completo.
Pseudonimização	Trocar CPF por identificador aleatório ou hash controlado.
Generalização	Converter idade exata em faixa etária.
Perturbação	Adicionar ruído quando a precisão individual não é necessária.
Agregação	Publicar resultados por município, escola ou grupo, não por pessoa.

Exemplo de anonimização simples

```

pessoas = pd.DataFrame({
    "nome": ["Joao", "Maria"],
    "idade": [25, 30],
    "cidade": ["Sao Paulo", "Rio de Janeiro"],
    "cpf": ["123.456.789-00", "987.654.321-00"]
})

pessoas_anon = (
    pessoas
    .assign(
        id_pessoa=lambda df: range(1, len(df) + 1),
        faixa_etaria=lambda df: pd.cut(
            df["idade"], bins=[0, 29, 39, 120], labels=["ate_29", "
                                                                30_39", "40_mais"]
        )
    )
    .drop(columns=["nome", "cpf", "idade"])
)

```

Checklist da base analítica final

- 1 A unidade de observação está explícita.
- 2 A chave é única ou a duplicidade está justificada.
- 3 Tipos de dados estão corretos.
- 4 Variáveis essenciais têm baixa ausência ou tratamento documentado.
- 5 Junções foram auditadas com `indicator` e `validate`.
- 6 Indicadores derivados foram calculados com fórmulas claras.
- 7 Agregações usam pesos quando substantivamente necessário.
- 8 A base final tem dicionário de variáveis e critérios de exclusão.

Onde estamos na aula?

Missão aplicada da aula

Problema

Construir uma base municipal para investigar como o desempenho educacional se relaciona com condições sociais e territoriais.

Municípios com maior vulnerabilidade social apresentam IDEB menor? A relação muda por região ou porte populacional?

Base 1

IDEB municipal

- município
- rede
- ano
- IDEB

Base 2

DTB/IBGE

- código municipal
- UF
- região
- nome oficial

Base 3

Atlas social

- pobreza
- analfabetismo
- Gini
- renda e IDHM

Checkpoint 1: transformar IDEB de largo para longo

```

ideb_long = (ideb
    .melt(
        id_vars=["SG_UF", "CO_MUNICIPIO", "NO_MUNICIPIO", "REDE"],
        var_name="ano",
        value_name="ideb")
    .assign(
        ano=lambda df: df["ano"].str.extract(r"(\d{4})").astype(int),
        sg_uf=lambda df: df["SG_UF"].str.strip(),
        rede=lambda df: df["REDE"].str.lower().str.strip(),
        co_municipio=lambda df: df["CO_MUNICIPIO"].astype(int)))
    
```

Perguntas para a turma

- 1 Depois do melt, o que representa uma linha?
- 2 A chave é apenas co_municipio + ano?
- 3 Por que rede precisa entrar na chave?

Checkpoint 2: validar chaves antes do merge

```
def diagnostico_chave(df, chave, nome):
    return pd.Series({
        "base": nome,
        "linhas": len(df),
        "chaves_unicas": df[chave].drop_duplicates().shape[0],
        "duplicadas": df.duplicated(chave).sum(),
        "ausentes_na_chave": df[chave].isna().any(axis=1).sum()})

pd.DataFrame([
    diagnostico_chave(ideb_long, ["co_municipio", "rede", "ano"], "ideb"),
    diagnostico_chave(dtb, ["co_municipio"], "dtb"),
    diagnostico_chave(atlas_2010, ["co_municipio"], "atlas_2010")])
```

Mini-entrega 1

Uma tabela de diagnóstico das chaves com: linhas, chaves únicas, duplicadas e ausentes.

Checkpoint 3: auditar cobertura com indicator=True

```
auditoria = (
    dtb
    .merge(
        ideb_long,
        on="co_municipio",
        how="outer",
        validate="1:m",
        indicator=True
    )
    auditoria["_merge"].value_counts()
```

Discussão rápida

- O que significa left_only?
- O que significa right_only?
- Para a análise final, faz mais sentido inner, left ou outer?

Checkpoint 4: construir a base analítica

```
df = (  
    dtb  
    .assign(  
        co_regiao=lambda x: x.co_municipio.astype(str)  
        .str[:1].astype(int),  
        no_regiao=lambda x: x.co_regiao.map({  
            1: "Norte", 2: "Nordeste", 3: "Sudeste",  
            4: "Sul", 5: "Centro-Oeste"})  
    ).merge(ideb_long, on="co_municipio", how="inner", validate="1:m")  
    .merge(atlas_2010, on="co_municipio", how="left", validate="m:1"))
```

Pergunta crítica

Ao juntar indicadores sociais de 2010 com IDEB de vários anos, estamos medindo associação contemporânea ou usando uma linha de base social?

Código quebrado: por que esse merge falha ou engana?

```
# Erro proposital: chaves com tipos diferentes e unidade mal definida
ideb_errado = ideb_long.assign(
    co_municipio=lambdax: x.co_municipio.astype(str))

base_errada = dtb.merge(ideb_errado, on="co_municipio", how="inner")
```

Tarefa de depuração

- 1 Identifique o erro de tipo.
- 2 Corrija a chave.
- 3 Acrescente validate.
- 4 Mostre quantas linhas mudaram antes e depois.

Correção esperada: merge auditável

```
ideb_ok = ideb_long.assign(co_municipio=lambda x: x.co_municipio.
                           astype(int))

df = (dtb
      .assign(co_municipio=lambda x: x.co_municipio.astype(int))
      .merge(
          ideb_ok,
          on="co_municipio",
          how="inner",
          validate="1:m",
          indicator="merge_dtb_ideb"))
df["merge_dtb_ideb"].value_counts()
```

Ideia-força

O código correto não é apenas o que executa: é o que permite auditar se o resultado faz sentido.

Onde estamos na aula?

Desafio de 15 minutos: tabela de evidências

```
ANO_REF = df["ano"].max()
resumo = (
    df.query("ano == @ANO_REF")
    .assign(log_pop=lambda x: np.log1p(x["pop"]))
    .groupby("no_regiao", as_index=False)
    .agg(
        municipios=("co_municipio", "nunique"),
        ideb_mediano=("ideb", "median"),
        pobreza_mediana=("ppob", "median"),
        renda_mediana=("rdpc", "median"),
        idhm_mediano=("idhm", "median"),
        pop_total=("pop", "sum"))
    .sort_values("ideb_mediano"))
```

Entrega no final da aula

Uma tabela com 3 a 5 linhas de evidência e um parágrafo interpretativo: **o que os dados sugerem e qual limitação impede uma conclusão causal?**

Rubrica da mini-entrega

Critério	Evidência observável	Pontos
Chave e unidade	Define corretamente a unidade de observação e a chave da base final	0–1
Merge auditado	Usa <code>validate</code> ou <code>indicator</code> e interpreta perdas/ausências	0–1
Manipulação	Usa corretamente <code>melt</code> , <code>merge</code> , <code>groupby</code> e variáveis derivadas	0–1
Interpretação	Apresenta evidência substantiva e reconhece a limitação temporal/causal	0–1

Critério mínimo de sucesso

A entrega deve permitir reproduzir a base final e entender por que cada junção foi feita.

Onde estamos na aula?

Síntese operacional

- 1 Comece pela pergunta, não pelo comando.
- 2 Defina a unidade de observação.
- 3 Valide a chave antes de integrar bases.
- 4 Padronize nomes, tipos e formatos.
- 5 Use concat para empilhar bases equivalentes.
- 6 Use merge para enriquecer bases com chaves comuns.
- 7 Audite perdas, duplicidades e ausências.
- 8 Use groupby para transformar registros em indicadores.
- 9 Use pivot_table e melt para reorganizar formatos.
- 10 Entregue uma tabela final que responda à pergunta inicial.

Leituras recomendadas

- Wickham, H. (2014). *Tidy Data*. Journal of Statistical Software.
- McKinney, W. (2022). *Python for Data Analysis*. O'Reilly.
- VanderPlas, J. (2017). *Python Data Science Handbook*. O'Reilly.
- Provost, F.; Fawcett, T. (2013). *Data Science for Business*. O'Reilly.
- CRISP-DM Consortium (2000). *CRISP-DM 1.0: Step-by-step data mining guide*.